COMP1011 Individual Project

Habit Tracker: A Command-Line Habit Tracking System

Submitted by: YIMI Supphakon

Student ID: 25091713d

1. Project Objectives

The Habit Tracker is a command-line interface (CLI) application written in C++ that allows users to build and maintain positive daily habits. The program targets students and individuals who prefer a lightweight, terminal-based tool with no external dependencies beyond the C++ standard library.

The key objectives of this project are:

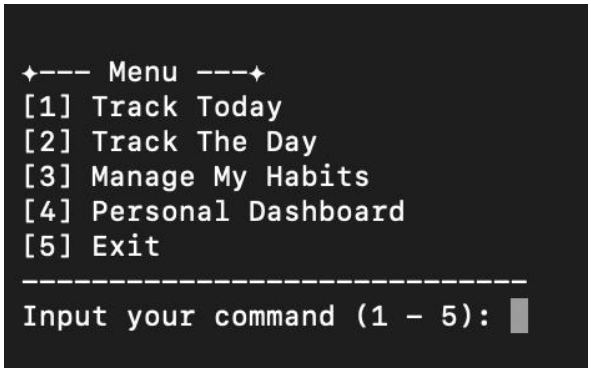
- Allow users to define, add, and delete personal habits.
- Enable daily habit check-in, allowing users to mark habits as completed or not for any given date.
- Persist tracking data across sessions using a CSV file (`habits.csv`), so progress is never lost.
- Provide a heat-map calendar visualization in the terminal, giving users a quick visual overview of their consistency over any given month.
- Present a personal dashboard with streak statistics, weekly completion rates, and best/worst habit highlights, giving users an analysis of their habit completion.

2. Key Features

This section describes the six core features of the Habit Tracker, each of which involves multiple interconnected functions working together to deliver a complete piece of functionality.

Note: For each function, variable, and file, is specifically written with a grey background and a distinct font. And the **core variables** are defined in the heading "Core Data Structures Summary."

2.1 Main Menu Navigation



The main menu is the central hub of the application, implemented as an infinite loop inside `main()`. On every iteration, `ClearScreen()` wipes the terminal using an ANSI escape sequence

before PrintMenu() shows the five options for users to navigate. This ensures the display is always clean and free of leftover output from the previous action.

Input validation is handled by `IsValidCmd(cmd, end)`, a general-purpose validator used throughout the application. It checks that the input is either the letter ‘q’ or a number within the range `[1, end]`, rejecting empty strings, non-numeric characters, and out-of-range values. If the user enters an invalid command, `PrintMenu()` is called again with a boolean flag set to true, which causes it to print an error message above the options before asking the user to input the command again.

Once a valid command is received, a conditional block directs the program to the corresponding feature function: `TrackToday()`, `TrackTheDay()`, `ManageMyHabits()`, or `PersonalDashboard()`. Entering ‘5’ breaks the loop and ends the program. After any feature function returns (user input ‘q’), the loop then immediately resumes from the top (clearing the screen and re-drawing the menu), so the user always lands back at the main menu.

2.2 Daily Habit Tracking

```
==================================================
        📅 Daily Checklist - April 13, 2026
==================================================

✨ Habits Status ✨
[/]  Wake up 8 A.M.
[/]  Drink 8 glasses of water
[/]  Journaling
[ ]  Meditation for 5 mins

--------------------------------------------------
[!] Entry Mode: Type 'y' for Success, 'n' for Miss.

> [1/4] Did you complete Wake up 8 A.M. ?: y

> [2/4] Did you complete Drink 8 glasses of water ?: n

> [3/4] Did you complete Journaling ?:
```

This feature allows the user to track their habit for the **current day**. When option [1] is selected from the main menu, `TrackToday()` is called, and the program enters an interactive entry session guided by an integer `statusTrackToday` flag with three states:

- State 1 (Full Entry): The user is prompted to answer ‘y’ or ‘n’ to track their completion for every habit, one by one.
- State 2 (Check Remaining): Only habits not yet marked as done are shown, allowing the user to top up completions later in the day without re-entering already completed ones.
- State 0 (View Only): All habits have been entered; the user can only view the status or choose to re-edit.

After entry, the screen is refreshed, and `PrintDateAndHabitStatus()` displays the current checklist with a checkmark '`[/]`' or blank '`[ ]`' for each habit. The function reads from a hash map `tracking_data[habit][date]` to show the completion of each habit.

If not all habits are completed, the user is presented with an option to re-edit all entries (state 1) or check remaining habits (state 2). This state machine approach avoids forcing the user to re-enter everything when they simply want to add one more completion later in the day. After any change, `SaveData()` is called automatically to save the updated state to a file.

2.3 Past Date Tracking

```
[q] Back to Menu
What's a date you want to track? (year-month-day, e.g. 2026-05-18) :
```

This feature lets the user log or edit habit completions for any past date. When option [2] is selected from the main menu, the user is prompted to type a date in year-month-day format (e.g. 2026-04-10).

The date input goes through a multi-step validation pipeline before the next process:

- `IsValidDate()` first checks that all characters are digits or hyphens, then parses the date into year, month, and day components, checks each against its valid range, and finally rejects any date in the future by comparing it against the current system date using `DateToNum()`.
- `DateFormatting()` then normalises the accepted string into a consistent zero-padded format (e.g. '2026-4-7' becomes '2026-04-07'), preventing duplicate keys in the data structures.
- `BinarySearchDate()` checks whether the normalised date already has an entry in `dates`. If not, the date is inserted, and `DateSort()` re-sorts the vector by integer value of dates (`DateToNum()`) to maintain the ordering required by binary search and streak calculation.

Once the date is validated and registered, the same entry flow as `TrackToday()` is used: the user answers 'y' or 'n' for each habit, and the results are stored in 2 hash maps (`tracking_data` and `progressBydate`). The user may then edit the same day again or jump straight to tracking another date, making it easy to fill in several missed days in one session.

2.4 Habit Management

```
✨ Habits Status ✨
[/]  1. Wake up 8 A.M.
[ ]  2. Drink 8 glasses of water
[/]  3. Journaling
[/]  4. Meditation for 5 mins

--------------------------------------------------
[1] Add new habit · [2] Remove a habit · [q] Back to Menu
Input command:
```

This feature lets the user modify a list of habits. When option [2] is selected from the main menu, the program will present two sub-options: adding a new habit or removing an existing one.

- **Adding a habit**: The user types the name of the new habit. A confirmation prompt ('y/n') is shown to prevent accidental additions. The name is appended to the string vector `habits`. Because `tracking_data` (hash map) is keyed by habit name and date, untracked dates for the new habit simply return false by default from the unordered_map.
- **Removing a habit**: The user selects a habit by its displayed number. The corresponding habit is erased from the `habits` vector by using `habits.erase(habits.begin() + index)`. Historical tracking data for the removed habit remains in `tracking_data` in memory but is no longer written to the CSV on the next `SaveData()` call, since `SaveData()` iterates only over the current string vector `habits`. This ensures the CSV stays clean without requiring an explicit clearing of old data.

2.5 Personal Dashboard

```
streak                    today's progress          weekly avg
1 day(s)                  6/4                       21%
personal best: 0          150% complete             ↑ 21% vs last week

  TODAY'S HABITS
/ Wake up 8 A.M.                                      done
X Drink 8 glasses of water                            missed
/ Journaling                                          done
/ Meditation for 5 mins                               done

  WEEKLY COMPLETION
Wake up 8 A.M.                  ■■■ ------------------ 14%  | 1d streak
Drink 8 glasses of water        ------------------ 0%   | 0d streak
Journaling                      ■■■ ------------------ 14%  | 1d streak
Meditation for 5 mins           ■■■ ------------------ 14%  | 1d streak

BEST HABIT                                NEEDS WORK
Meditation for 5 mins                     Drink 8 glasses of water
14% this week · 1d streak                 0% this week · 0d streak

April, 2026
Sun Mon Tue Wed Thu Fri Sat
            1   2   3   4
5   6   7   8   9   10  11
12  13  14  15  16  17  18
19  20  21  22  23  24  25
26  27  28  29  30

 less   ■ ■ ■ ■   more

-------------------------------------------------------------------
[1] Previous Month · [q] Back to Menu
Input command:
```

The Personal Dashboard is for users' habits data visualization, combining streak statistics, weekly completion rates, a best/worst habit comparison, and a heat-map calendar into a single view.

Streak Calculation:

`CalculateStreak()` computes both a per-habit streak and an overall streak by iterating over `dates`. The helper `IsValidStreak(first_date, second_date)` returns true only when the second date is exactly one calendar day after the first, correctly handling month-end and year-end boundaries. For each consecutive pair of dates, if the dates are adjacent and the habit (or any habit for the overall streak) was completed on both, the streak will be increased; otherwise, it will reset to 1. However, if neither today nor yesterday has any recorded progress, the displayed streak is set to 0. The personal best (`bestStreak`) is updated at every step if a longer streak is found to find the longest streak.

Weekly Averages:

`WeeklyAvg()` calculates backwards from the current date to the most recent Monday using `Yesterday()` and `getDayOfWeek()`, summing `progressBydate` for each day, then divides by (7 x total habits) to produce an overall weekly completion percentage. `WeeklyHabitAvg()` applies the same method but sums `tracking_data[habit][date]` for a single habit, calculating a per-habit weekly rate. These percentages are used in the progress bar display and the this week and last week comparison.

*Note: `getDayOfWeek()` was written by **AI** using Zeller's Congruence (algorithm to find a day of the week).*

Best and Worst Habit Detection:

The dashboard scans all habits using `WeeklyHabitAvg()` to find the habit with the highest weekly rate (best) and the lowest (worst). These are highlighted in blue and red respectively, along with their weekly percentage and current streak.

Heat-Map Calendar:

`PrintCalendar()` shows a full monthly calendar in the terminal. It first calls `getDayOfWeek()` to determine which column the 1st of the month is located, then prints blank offset cells accordingly. For each day, `HeatColor()` maps the ratio of `progressBydate[date]` to total habits onto one of five ANSI RGB background colours (gray for 0%, light green to dark green for increasing completion). An explanation of colour is printed beneath the grid.

The dashboard supports month navigation: the user can step back to previous months via '*Previous Month*' option, and it correctly handles January to December across years. '*This Month*' option reappears once the user has navigated away from the current month.

2.6 CSV Persistence

User data is stored in a plain-text CSV file called `habits.csv`. This format was chosen because it is human-readable, easy to implement, easy to export to spreadsheet tools, and requires no external libraries.

`ReadData()` is called once at startup. It first uses `IsValidFile()` to check whether `habits.csv` exists and is readable. If the file is not found, the program starts with an empty state. If the file exists, all habit names in the first line are extracted into the habits vector. Each subsequent line is then parsed column by column: the date string is stored in `dates` along with its integer representation from `DateToNum()`, and each habit column value ('1' or '0') is written into `tracking_data[habit][date]`. The `progressBydate` counter for each date is also accumulated during this process.

`SaveData()` is called automatically after every user action that modifies data (tracking, adding or removing habits). It reopens `habits.csv` in overwrite mode, writes the header row, sorts dates chronologically by integer key (`dates`), then writes one row per date.

2.7 Date Management

The Date Management is the utility tool that all other features rely on.

Representation and Parsing:

Dates are stored internally as strings in the normalised format YYYY-MM-DD. `DateToStruct()` parses such a string into a `struct Date {y, m, d}` by scanning character by character and splitting on '-'. `DateFormatting()` ensures any input date (which may omit leading zeros) is converted to the normal form. `DateToMDY()` converts the internal format to a readable form (e.g. 'April 13, 2026') for display purposes.

Integer Key (DateToNum):

`DateToNum()` encodes a date string as a single integer using the formula (year x 366) + (month x 31) + day. This encoding is increasing for all valid dates, enabling fast numeric comparisons and binary search for dates. The integer keys are stored alongside date strings in the `vector<pair<string, int>> dates` and are used throughout the application for sorting, searching, and streak validation.

Date Arithmetic:

`Yesterday()` computes the previous calendar date from any given date string, handling three cases: stepping back within a month (decrement day), crossing a month boundary (roll back to the last day of the previous month using `daysInMonth[]`), and crossing a year boundary (roll back to December 31 of the previous year). `IsLeapYear()` applies the standard Gregorian leap-

year rules (divisible by 400, or divisible by 4 but not 100) and is used by `Yesterday()` when rolling back from March 1st.

Current Date:

`CurrentDate()` uses the C standard library `time()` and `localtime()` functions to retrieve the current system date, returning it either as a normalised YYYY-MM-DD string or as a formatted, readable string depending on the boolean parameter passed.

3. Examples and User Manual

3.1 Launching the Application

Compile and run the program:

```
| g++ -o habit_tracker main.cpp date.cpp fileManagement.cpp screen.cpp
| ./habit_tracker
```

On first launch, if no `habits.csv` file is found, the application starts with an empty habit list. The main menu is displayed immediately.

Note: if `./habit_tracker` *does not work, try* `habit_tracker`

3.2 Main Menu

Option	Description
[1] Track Today	Mark habits as done or not for the current date.
[2] Track The Day	Retrospectively mark habits for a past date entered by the user.
[3] Manage Habits	Add or delete habit names from the tracked list.
[4] Dashboard	View streak stats, weekly completion, and the heat-map calendar.
[5] Save	Quit the program.

Input: the corresponding number of a desired option.

3.3 Track Today

1. **Input:** ‘`y`’ for a success habit or ‘`n`’ for a missed habit.
2. **Input:** ‘1’ to re-track today’s habit completion, ‘2’ to track the remaining habits that are still uncompleted, and ‘`q`’ to go back to the main menu.

3.4 Past Date Tracking

1. **Input:** the date (YYYY-MM-DD) when the user wants to track.
2. **Input**: ‘`y`’ for a success habit or ‘`n`’ for a missed habit.
3. **Input:** ‘1’ to re-track today’s habit completion, and ‘`q`’ to go back to the main menu.

3.5 Personal Dashboard

Input: ‘1’ for `Previous Month` option to display the heat-map calendar of the previous month. If the past month is currently displayed, `This Month` option will appear for users to show the heat-map calendar of the current month.

4. AI Tool Disclosure

The generated code is clearly marked in the source file with a comment `//Gen AI code`.

- The function `getDayOfWeek()` in `date.cpp` was developed with the assistance of an AI tool. This function implements Zeller's Congruence to compute the day of the week for a given date. The generated code is clearly marked in the source file with a comment.
- The function `ClearPreviousLines()` in `screen.cpp`, which uses ANSI escape sequences to erase terminal lines, was generated with AI.
- The colour palette array in `main.cpp` (the vector of ANSI RGB colour strings used in the weekly completion bar chart) was also generated with AI.

All other functions, including data management, streak calculation, binary search, date arithmetic, calendar rendering, and the overall program architecture, were written independently by the **author**.

No open-source projects were used as a base. The AI-generated segments are utility functions that are not part of the core logic of the application.

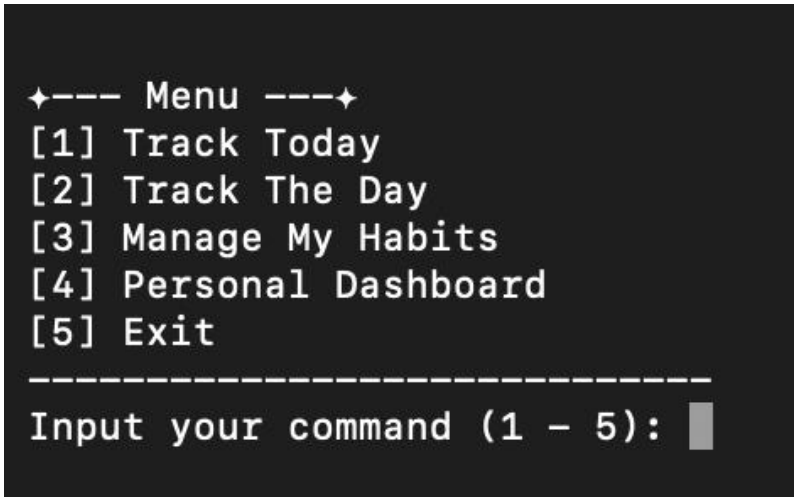
5. Core Data Structures Summary

Variable	Type	Purpose
habits	vector<string>	Ordered list of habit names; defines column order in CSV.
dates	vector<pair<string,int>>	All tracked dates with integer key for sorting and binary search.
tracking_data	unordered_map<string, unordered_map<string,bool>>	Two-level map: habit name -> date -> completion status.
progressBydate	unordered_map<string,int>	Number of habits completed per date.
bestStreak	Streak struct	All-time personal best streaks, both overall and per habit.

6. Screenshots of Key Results

The following screenshots demonstrate the application running in a terminal environment.

[Screenshot 1] Main menu as displayed on launch.



[Screenshot 2] Track Today screen showing habit entry and toggle interface.

```
==================================================
        📝 Daily Checklist - April 13, 2026
==================================================

✨ Habits Status ✨
[ ]  Wake up 8 A.M.
[ ]  Drink 8 glasses of water
[ ]  Journaling
[ ]  Meditation for 5 mins

--------------------------------------------------
[!] Entry Mode: Type 'y' for Success, 'n' for Miss.

> [1/4] Did you complete Wake up 8 A.M. ?: y

> [2/4] Did you complete Drink 8 glasses of water ?: n

> [3/4] Did you complete Journaling ?: y

> [4/4] Did you complete Meditation for 5 mins ?:
```

[Screenshot 3] Past date tracking with date validation.

```
[q] Back to Menu
What's a date you want to track? (year-month-day, e.g. 2026-05-18) : 2026-03-17
```

[Screenshot 4] Past date tracking showing toggle interface after inputted valid date.

```
==================================================
        📝 Checklist on - March 17, 2026
==================================================

✨ Habits Status ✨
[ ]  Wake up 8 A.M.
[ ]  Drink 8 glasses of water
[ ]  Journaling
[ ]  Meditation for 5 mins

--------------------------------------------------
[!] Entry Mode: Type 'y' for Success, 'n' for Miss.

> [1/4] Did you complete Wake up 8 A.M. ?: y

> [2/4] Did you complete Drink 8 glasses of water ?: y

> [3/4] Did you complete Journaling ?: n
```

[Screenshot 5] Habit Management showing a list of habits and commands to modify the user's habits.

```
✨ Habits Status ✨
[/]  1. Wake up 8 A.M.
[ ]  2. Drink 8 glasses of water
[/]  3. Journaling
[/]  4. Meditation for 5 mins

--------------------------------------------------
[1] Add new habit · [2] Remove a habit · [q] Back to Menu
Input command:
```

[Screenshot 6] Personal Dashboard showing streak stats, weekly completion bars, and best/worst habits.

```
streak                    today's progress        weekly avg
3 day(s)                  2/4                     17%
personal best: 4          50% complete            ↓ 22% vs last week

  TODAY'S HABITS
/ Wake up 8 A.M.                                    done
X Drink 8 glasses of water                          missed
/ Journaling                                        done
X Meditation for 5 mins                             missed

  WEEKLY COMPLETION
Wake up 8 A.M.                  28%  | 3d streak
Drink 8 glasses of water        0%   | 0d streak
Journaling                      28%  | 3d streak
Meditation for 5 mins           14%  | 1d streak

BEST HABIT                          NEEDS WORK
Journaling                          Drink 8 glasses of water
28% this week · 3d streak           0% this week · 0d streak

April, 2026
Sun Mon Tue Wed Thu Fri Sat
            1   2   3   4
5   6   7   8   9   10  11
12  13  14  15  16  17  18
19  20  21  22  23  24  25
26  27  28  29  30

 less                   more

--------------------------------------------------------------------
[1] Previous Month · [q] Back to Menu
Input command:
```

[Screenshot 7] habits.csv file content after saving data.

```csv
date,Wake up 8 A.M.,Drink 8 glasses of water,Journaling,Meditation for 5 mins
2026-02-16,0,0,1,1
2026-03-11,1,1,1,1
2026-04-01,1,0,0,0
2026-04-02,1,0,0,1
2026-04-03,1,0,1,1
2026-04-04,1,0,1,1
2026-04-06,1,0,1,0
2026-04-07,1,1,0,0
2026-04-08,1,1,0,0
2026-04-09,1,1,0,0
2026-04-11,1,0,1,1
2026-04-12,1,0,1,1
2026-04-13,1,1,0,1
```

Github repo and Git commit log for reference